3.1.2. Giá trị của kiểm thử tĩnh (Value of Static Testing)

Lợi ích chính 🎯

Kiểm thử tĩnh (Static Testing) phát hiện **khuyết tật (defects)** ngay từ giai đoạn đầu SDLC, thực hiện nguyên tắc "kiểm thử sớm" (early testing). Khác với kiểm thử động, nó có thể tìm ra những lỗi mà kiểm thử động bỏ sót.

Ví dụ thực tế: Review tài liệu yêu cầu có thể phát hiện yêu cầu mâu thuẫn ("Hệ thống phải xử lý trong 2 giây" và "Hệ thống phải xử lý 10,000 giao dịch/giây"), trong khi kiểm thử động chỉ phát hiện vấn đề khi code đã chạy.

Ba giá trị cốt lõi

1. Xây dựng niềm tin và hiểu biết chung (Building Confidence & Shared Understanding)

- Xác minh yêu cầu phản ánh đúng nhu cầu thực tế
- Tạo sự đồng thuận giữa các bên liên quan sớm
- Cải thiện giao tiếp trong team

Ví dụ: Trong buổi review yêu cầu, BA, Dev và Tester cùng phát hiện "đăng nhập" chưa định nghĩa rõ có hỗ trợ OAuth hay chỉ username/password.

2. Giảm chi phí dự án (Reduced Project Cost) 💰

- Chi phí sửa lỗi tăng theo cấp số nhân qua các giai đoạn
- Review mất công nhưng ROI rất cao

Con số thực tế: Sửa lỗi ở giai đoạn yêu cầu tốn 1x, ở giai đoạn code tốn 10x, ở production tốn 100x chi phí.

3. Phân tích mã hiệu quả (Code Analysis Efficiency)

- Phát hiện một số lỗi nhanh hơn kiểm thử động
- Đo lường chất lượng không cần chạy code (maintainability, complexity)

Ví dụ: Static analysis tool tự động phát hiện biến chưa khởi tạo, mã không bao giờ chạy (unreachable code) - những lỗi khó test bằng cách chạy thử.

Các khuyết tật điển hình phát hiện được

Loại khuyết tật	Mô tả	Ví dụ cụ thể
Khuyết tật yêu cầu (Requirements defects)	Không nhất quán, mơ hồ, mâu thuẫn, thiếu sót, không chính xác, trùng lặp	"Hệ thống phải user-friendly" (mơ hồ), hoặc yêu cầu #5 và #12 mâu thuẫn nhau về quy trình thanh toán
Khuyết tật thiết kế (Design defects)	Cấu trúc database kém, module hóa thấp	Database không chuẩn hóa (denormalized) gây trùng lặp dữ liệu, hoặc một class xử lý quá nhiều trách nhiệm
Khuyết tật mã nguồn (Code defects)	Biến không xác định, chưa khai báo, unreachable code, code trùng lặp, độ phức tạp cao	`int result; return result;` (chưa gán giá trị), hoặc code sau lệnh `return` không bao giờ chạy

Sai lệch tiêu chuẩn (Standard deviations)	Vi phạm coding conventions	Đặt tên biến `int X1;` thay vì `int userId;`, hoặc không tuân thủ quy ước camelCase
Lỗi giao diện (Interface defects)	Sai số lượng, kiểu dữ liệu, thứ tự tham số	Function định nghĩa `calc(int a, String b)` nhưng gọi `calc("text", 5)`
Lỗ hổng bảo mật (Security vulnerabilities)	Buffer overflow, SQL injection	Code không check độ dài input trước khi copy vào buffer 256 bytes
Thiếu độ bao phủ (Coverage gaps)	Test case thiếu cho tiêu chí chấp nhận	Requirement có 5 điều kiện nhưng chỉ có test case cho 3 điều kiện

Điểm khác biệt quan trọng ⚡

Kiểm thử động: Phát hiện **failures** (lỗi khi chạy) → Cần debug để tìm **defect** (nguyên nhân)

Kiểm thử tĩnh: Phát hiện trực tiếp **defects** → Không cần debug

Ví dụ:

- Động: App crash → phải trace log, debug để tìm biến null
- Tĩnh: Code review thấy ngay `user.getName()` mà không check `user != null`

Kết luận: Static testing là "an toàn hơn chữa bệnh" - đầu tư vào review và analysis công phu ban đầu sẽ tiết kiệm gấp nhiều lần chi phí sửa lỗi sau này.

Một số ghi chú

- Tài liệu dịch từ sách gốc ISTQB Foudation v4.0.1 sang tiếng Việt, nhằm mục đích giảm rào cản tiếp cận các kiến thức về testing tới cộng đồng tester Việt Nam nói chung và các anh chị em muốn tìm hiểu về testing nói riêng
- Tài liệu cố gắng dịch nhiều nhất các từ tiếng Việt để bạn đọc không phải tra từ điển trong quá trình đọc (VD: defects dịch là khuyết tật phần mềm).
- Dự án phi lợi nhuận, bạn có thể thoải mái sử dụng bản dịch, chia sẻ và sửa đổi nếu cần thiết.
- Có góp ý cho dự án, bạn có thể submit góp ý qua link này nha: https://go.betterbytesvn.com/sharing-documentation-feedback.
- Bản dịch được thực hiện bởi tác giả Đỗ Minh Phong. Bạn có thể gửi lời cảm ơn/feedback/ donate tới tác giả thông qua các hình thức:
  - Đăng ký kênh Youtube: https://www.youtube.com/@hoctest_com
  - Follow Fanpage: https://www.facebook.com/hoctest/
  - Tham gia group Playwright Việt Nam trên Facebook: https://www.facebook.com/groups/playwright.automation.test
  - Donate cho tác giả: https://academy.betterbytesvn.com/donate-cho-chung-minh/
  - Facebook cá nhân tác giả: https://www.facebook.com/dominhphong.18/

Xin chân thành cảm ơn bạn, vì đã quan tâm tới tài liệu ^^.